

Assignment No. 1**Title**

Implementation of Binary Tree and Recursive Tree Traversals (In-order, Preorder, and Post-order)

Problem Statement

Create a Binary Tree and implement the following recursive traversal techniques:

- Inorder Traversal
- Preorder Traversal
- Postorder Traversal

Display the nodes of the binary tree using each traversal method.

Scenario-Based Problem Statement**Library Catalog Navigation**

A university library stores book records in a hierarchical structure, where each book category is connected to its subcategories and individual books. To efficiently navigate and organize the catalog, the library represents the records using a Binary Tree.

Develop a Binary Tree to represent the library catalog and implement the following recursive traversal techniques:

- Inorder Traversal – Displays book categories in sorted navigation order.
 - Preorder Traversal – Displays the root category before its subcategories, useful for creating a catalog structure.
 - Postorder Traversal – Displays subcategories before the parent category, useful for safely deleting or archiving catalog entries.
-

Objective(s)

After completing this assignment, students will be able to:

1. Understand the concept and structure of a Binary Tree.
 2. Create a Binary Tree using linked representation.
 3. Implement recursive tree traversal algorithms.
 4. Differentiate between Inorder, Preorder, and Postorder traversals.
 5. Apply Binary Trees to solve real-world hierarchical problems.
-

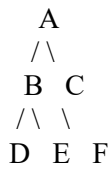
Theory**Binary Tree**

A Binary Tree is a non-linear data structure in which each node has at most two child nodes, known as the left child and the right child.

Each node consists of:

- Data
- Pointer to Left Child

- Pointer to Right Child



Recursive Traversals

Tree traversal means visiting every node exactly once.

Traversal Method	Visiting Sequence	Traversal Rule	Memory Trick	Applications
Inorder Traversal	Left → Root → Right (LNR)	Visit the left subtree first, then the root node, and finally the right subtree.	LNR (Left-Node-Right)	Binary Search Tree (BST) traversal, sorting elements, expression trees
Preorder Traversal	Root → Left → Right (NLR)	Visit the root node first, then the left subtree, and finally the right subtree.	NLR (Node-Left-Right)	Creating or copying trees, directory structures, prefix expression
Postorder Traversal	Left → Right → Root (LRN)	Visit the left subtree first, then the right subtree, and finally the root node.	LRN (Left-Right-Node)	Deleting trees, expression evaluation, postfix expression

Binary Tree Node Structure

```

struct Node
{
    int data;
    Node *left;
    Node *right;
};

```

Complexity Analysis

Traversal Time Space

Inorder $O(n)$ $O(h)$

Preorder $O(n)$ $O(h)$

Postorder $O(n)$ $O(h)$

where

- n = Number of nodes
- h = Height of tree

Algorithm

Algorithm for Creating Binary Tree

1. Create a new node.
2. Read node data.
3. If data is NULL, return NULL.
4. Create left subtree recursively.
5. Create right subtree recursively.
6. Return root node.

Algorithm for Inorder Traversal	Algorithm for Preorder Traversal	Algorithm for Postorder Traversal
If root is NULL return Traverse Left Subtree Visit Root Traverse Right Subtree	If root is NULL return Visit Root Traverse Left Subtree Traverse Right Subtree	If root is NULL return Traverse Left Subtree Traverse Right Subtree Visit Root

C++ Program	Python Program
<pre>#include <iostream> using namespace std; struct Node { int data; Node *left, *right; Node(int value) { data = value; left = right = NULL; } }; Node* createTree() { int value; cout<<"Enter Data (-1 for NULL): "; cin>>value; if(value==-1) return NULL; Node *root=new Node(value); cout<<"Left child of "<<value<<endl; root->left=createTree();</pre>	<pre>class Node: def __init__(self, data): self.data = data self.left = None self.right = None def createTree(): value = int(input("Enter Data (-1 for NULL): ")) if value == -1: return None root = Node(value) print("Left child of", value) root.left = createTree() print("Right child of", value) root.right = createTree() return root def inorder(root): if root: inorder(root.left) print(root.data, end=" ") inorder(root.right) def preorder(root):</pre>

<pre> cout<<"Right child of "<<value<<endl; root->right=createTree(); return root; } void inorder(Node *root) { if(root==NULL) return; inorder(root->left); cout<<root->data<<" "; inorder(root->right); } void preorder(Node *root) { if(root==NULL) return; cout<<root->data<<" "; preorder(root->left); preorder(root->right); } void postorder(Node *root) { if(root==NULL) return; postorder(root->left); postorder(root->right); cout<<root->data<<" "; } int main() { Node *root; root=createTree(); cout<<"\nInorder : "; inorder(root); cout<<"\nPreorder : "; preorder(root); cout<<"\nPostorder : "; postorder(root); return 0; } </pre>	<pre> if root: print(root.data, end=" ") preorder(root.left) preorder(root.right) def postorder(root): if root: postorder(root.left) postorder(root.right) print(root.data, end=" ") root = createTree() print("\nInorder :", end=" ") inorder(root) print("\nPreorder :", end=" ") preorder(root) print("\nPostorder :", end=" ") postorder(root) </pre>
--	--

Q1. What is a Binary Tree?

Q2. Why is recursion used in tree traversal?

Q3. Compare recursive traversal with iterative traversal.

Q4. Enlist the Applications of Binary tree

Q5. What is the time and Space complexity of recursive traversals?

Snippet 1: Q. Why is the Traversal Output Missing?	Snippet 2: Q. Why Does This Program Crash?	Snippet 3: Q. Identify the traversal?
<pre>void inorder(Node *root) { if(root = NULL) return; inorder(root->left); cout << root->data << " "; inorder(root->right); }</pre>	<pre>void preorder(Node *root) { cout << root->data << " "; preorder(root->left); preorder(root->right); }</pre>	<pre>void traverse(Node *root) { if(root == NULL) return; traverse(root->left); traverse(root->right); cout << root->data << " "; }</pre>
Ans: -----	Ans: -----	Ans: -----

Conclusion

In this assignment, a Binary Tree was successfully created using linked representation, and the Inorder, Preorder, and Postorder recursive traversal techniques were implemented.